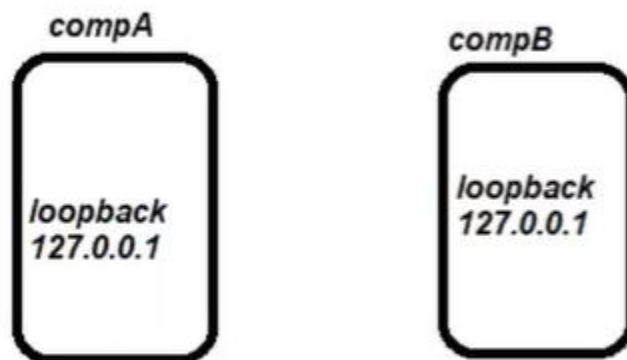


Networking: Source, destination, how packets transferred from source to destination and what is the route it is taking.

General Networking

When your computer is not connected to any network by default it contains a loopback IP address (127.0.0.1)

local (loopback) address: 127.0.0.1



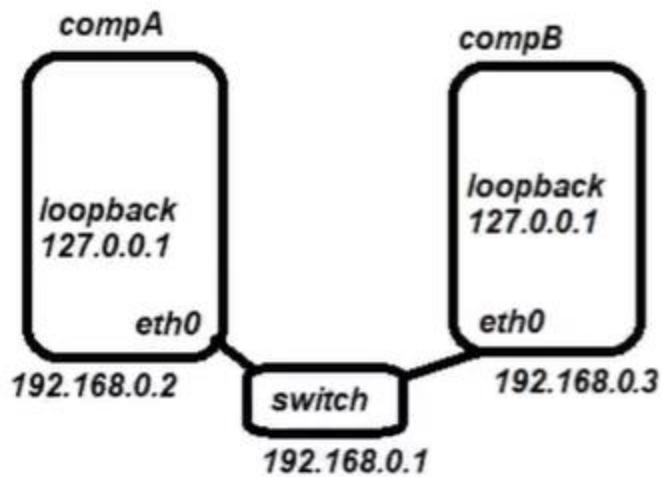
ifconfig

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 132 bytes 13488 (13.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 132 bytes 13488 (13.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

How to establish a communication between two computers?

Using switch (considered as a LAN network). Switch is the device which connects two computers.

When we connect the computer from switch (LAN) it creates an interface eth0 on every node, an IP address is generated in your computer allocated by the switch.



eth0: Interface at vm (IP address allocated by switch)

ens4: vm joins external network (GC).

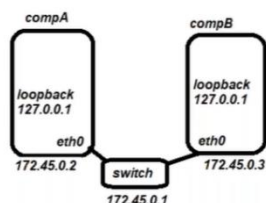
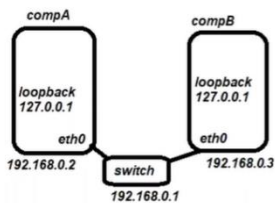
```

root@controlplane:~# ifconfig
ens4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 10.150.0.6 netmask 255.255.255.255 broadcast 0.0.0.0
    inet6 fe80::4001:aff:fe96:6 prefixlen 64 scopeid 0x20<link>
    ether 42:01:0a:96:00:06 txqueuelen 1000 (Ethernet)
    RX packets 1122 bytes 996020 (996.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 986 bytes 111282 (111.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

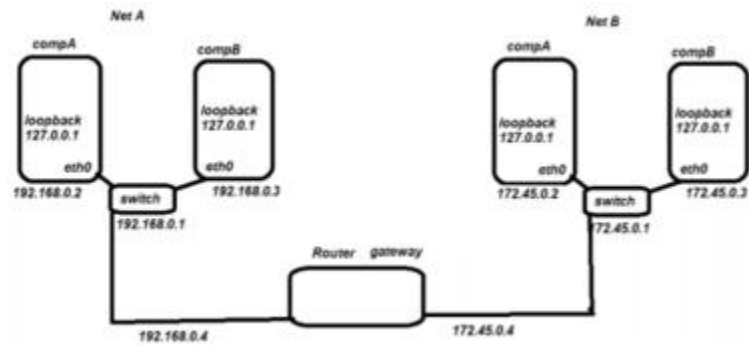
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 132 bytes 13488 (13.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 132 bytes 13488 (13.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

root@controlplane:~#
  
```

UC: We have another network available.



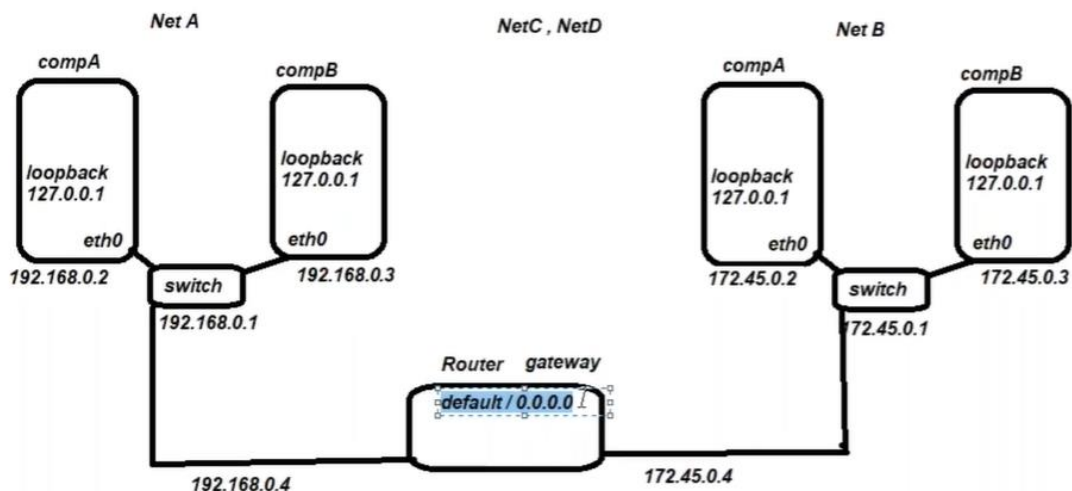
UC: How establish a communication between two different networks, we need to use different device called Router, using Router we can connect different networks Router connects multiple networks. This gets IP address against each network.



Router is acting as a gateway, from where it allows to connect to the different network.

UC: We have Network C, Network D but we don't know the IP addresses of that Networks, can we establish a communication with unknown networks also?

Yes, by entering default value 0.0.0.0 in Route table, which indicates to connect to unknown Network.



Docker Networking:

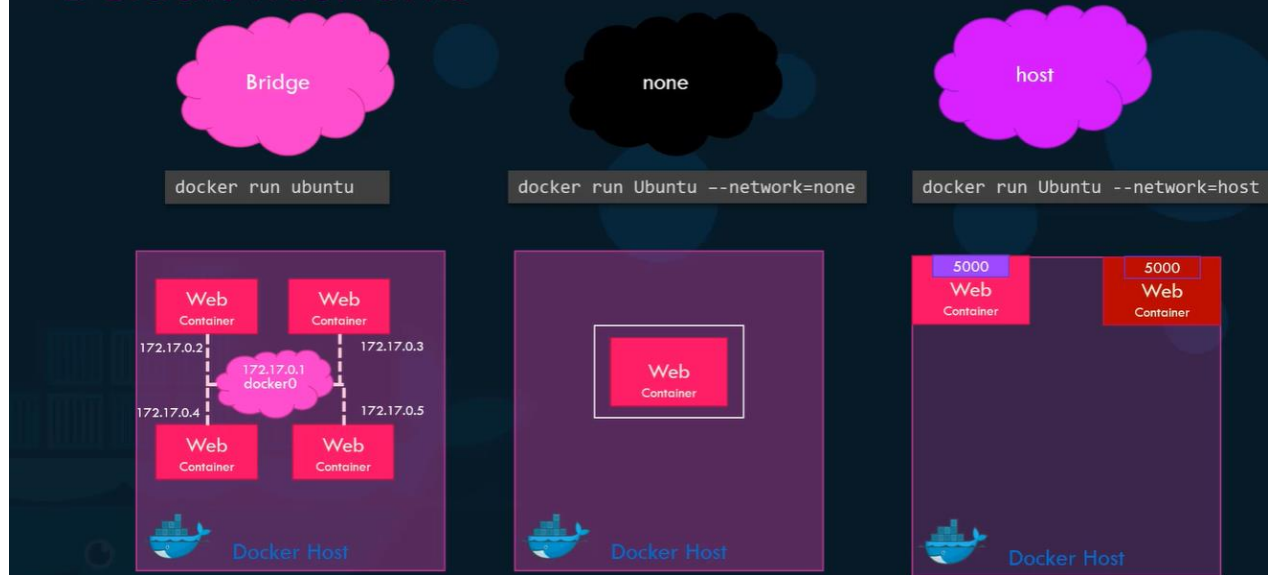
What are the network solutions provided by the docker?

docker network ls

```
root@controlplane:~# docker network ls
NETWORK ID   NAME      DRIVER  SCOPE
9d7d8349b98a bridge    bridge  local
fde09e4476bc host      host    local
9c763d38595e none      null    local
root@controlplane:~#
```

docker0 (CNM): container network model

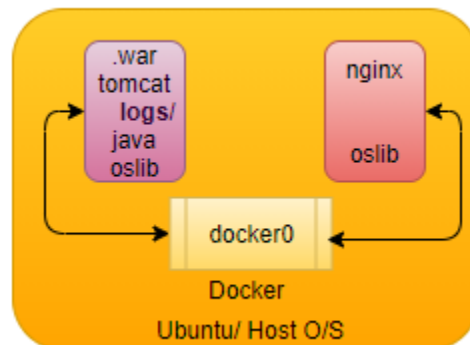
Default networks



docker inspect <network id>

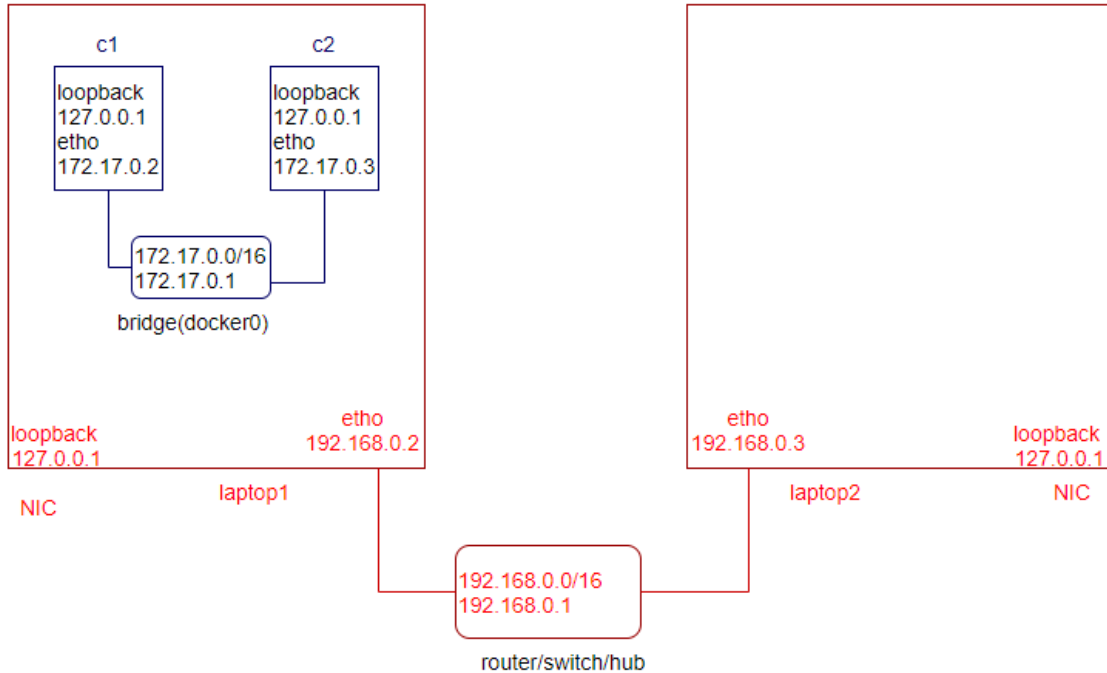
```
root@controlplane:~# docker inspect 9d7d8349b98a
[
  {
    "Name": "bridge",
    "Id": "9d7d8349b98ae742af6163bf9c1615b49dc9813549e575be91d97474421746f4",
    "Created": "2021-01-26T14:42:07.27326621Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.17.0.0/16"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {
      "com.docker.network.bridge.default_bridge": "true",
      "com.docker.network.bridge.enable_icc": "true",
      "com.docker.network.bridge.enable_ip_masquerade": "true",
      "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
      "com.docker.network.bridge.name": "docker0",
      "com.docker.network.driver.mtu": "1500"
    },
    "Labels": {}
  }
]
```

UC: We have two containers tomcat and nginx, can they communicate to each other by default?



docker network ls
 docker network inspect <bridge>
 ifconfig

loopback -local host
 etho-network
 docker0- virtual switch
 SDN-S/W defined network



How to containers are able to communicate to each other?

docker by default create a **virtual bridge or virtual switch (docker0)** for containers communication.

```
root@controlplane:~# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:68:f8:4c:62 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens4: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 10.150.0.6 netmask 255.255.255.255 broadcast 0.0.0.0
    inet6 fe80::4001:aff:fe96:6 prefixlen 64 scopeid 0x20<link>
    ether 42:01:0a:96:00:06 txqueuelen 1000 (Ethernet)
    RX packets 8623 bytes 126894125 (126.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 6217 bytes 600698 (600.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

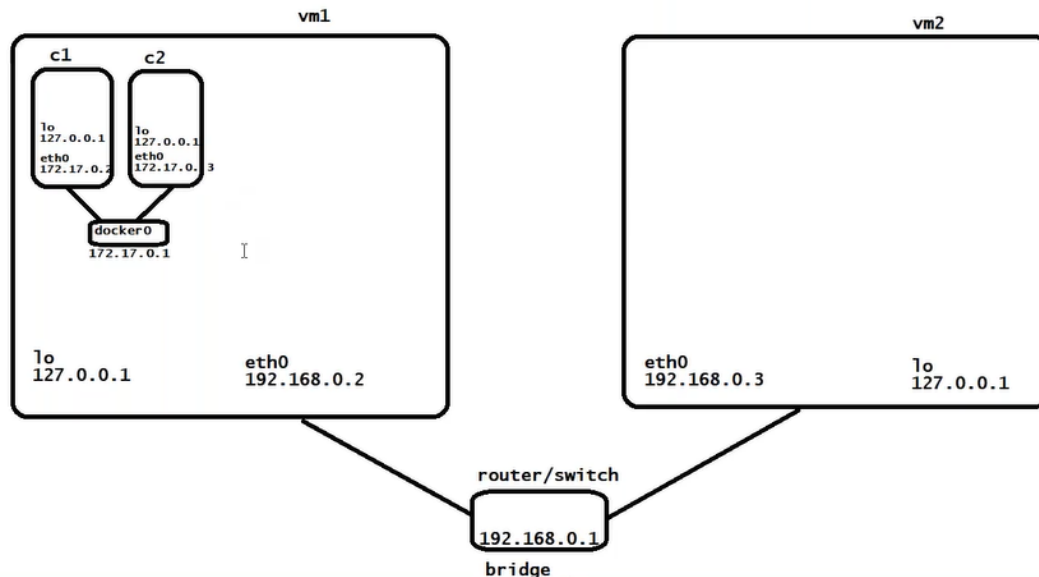
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 216 bytes 22989 (22.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 216 bytes 22989 (22.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

ens4: external network from where your vm gets IP addresses e.g. GCP.

io: Without connecting to any network every host has by default their own LOOPBACK address.

docker0: When we install the docker (virtual switch/interface), created by the docker to container communication within the same host, every container connected through the virtual interface veth0.

Whenever we create a container by default it joins the bridge (docker0) network. So that every container created in the same host is able to communicate to each other, each container created in the host connected to the virtual switch (docker0) through virtual interface (veth0) is created on each container and an IP address is assigned to veth0.



docker container run -d --name=cont1 lerndevops/alpine:sleep
 docker container run -d --name=cont2 lerndevops/alpine:sleep

```

root@controlplane:~# docker container run -itd --name=cont1 lerndevops/alpine:sleep
Unable to find image 'lerndevops/alpine:sleep' locally
sleep: Pulling from lerndevops/alpine
cbdbe7a5bc2a: Pull complete
8d139537a7e9: Pull complete
0ad5955f6131: Pull complete
Digest: sha256:89207cde1b4c2c6ba476818501065f5176c2d003a226941b9a226eada3925fa2
Status: Downloaded newer image for lerndevops/alpine:sleep
alf8631c80abea0e6f46615a08510091d841b128a52d40adc9acde8a8b7d679d
root@controlplane:~# docker container run -itd --name=cont2 lerndevops/alpine:sleep
d4cb2f98ff2272e44db74cb55a62a31749269a22bae91af64e5e611ef7319a10
root@controlplane:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS   NAMES
d4cb2f98ff22   lerndevops/alpine:sleep  "/app/sleep"            8 seconds ago   Up 7 seconds   -       cont2
alf8631c80ab   lerndevops/alpine:sleep  "/app/sleep"            16 seconds ago   Up 15 seconds   -       cont1
root@controlplane:~# docker exec cont1 hostname -i
172.17.0.2
root@controlplane:~# docker exec cont2 hostname -i
172.17.0.3
root@controlplane:~# docker exec cont1 ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3): 56 data bytes
64 bytes from 172.17.0.3: seq=0 ttl=64 time=0.197 ms
64 bytes from 172.17.0.3: seq=1 ttl=64 time=0.087 ms
64 bytes from 172.17.0.3: seq=2 ttl=64 time=0.095 ms
64 bytes from 172.17.0.3: seq=3 ttl=64 time=0.095 ms
^C
root@controlplane:~# docker exec cont2 ping 172.17.0.2
PING 172.17.0.2 (172.17.0.2): 56 data bytes
64 bytes from 172.17.0.2: seq=0 ttl=64 time=0.107 ms
64 bytes from 172.17.0.2: seq=1 ttl=64 time=0.110 ms
64 bytes from 172.17.0.2: seq=2 ttl=64 time=0.105 ms
^C
root@controlplane:~#
  
```

Virtual Switch (bridge network): docker0

Virtual Switch is just like another interface for the **host computer** to interact with the **container**.

Virtual Interface (Veth0): Whenever connect to the **bridge switch** there is a virtual interface is created on each container (veth0) and ip address assigned to veth0.

UC2: customization

How to create your own customized bridge?

docker network create --help

docker network create --driver bridge mybridge1

```
root@controlplane:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
9d7d8349b98a        bridge             bridge              local
fde05e4476bc        host              host                local
9c763d38595e        none              null                local
root@controlplane:~# docker network create --driver bridge mybridge
24e23771db15d989755ee5c01310730aa21000c0dd86fae3e0c067482252fe34
root@controlplane:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
9d7d8349b98a        bridge             bridge              local
fde05e4476bc        host              host                local
24e23771db15        mybridge           bridge              local
9c763d38595e        none              null                local
root@controlplane:~# docker inspect 24e23771db15
[
  {
    "Name": "mybridge",
    "Id": "24e23771db15d989755ee5c01310730aa21000c0dd86fae3e0c067482252fe34",
    "Created": "2021-01-26T15:41:55.260142814Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {}
  }
]
```

docker network create mybridge --driver bridge --subnet 192.168.0.0/16 --gateway 192.168.0.1

How to create container in customized network?

docker run -d --name=cont3 lerndevops/alpine:sleep # join default bridge

docker run -d --name=cont3 --network mybridge lerndevops/alpine:sleep

docker run -d --name=cont4 --network mybridge lerndevops/alpine:sleep

```
root@controlplane:~# docker run -d --name=cont3 --network mybridge lerndevops/alpine:sleep
ff4370d6cbb0507d866ade4c05e79ebe0934a7cd64861204336e7d72993f7ec3
root@controlplane:~# docker run -d --name=cont4 --network mybridge lerndevops/alpine:sleep
628c38c58626eef67b5093cfb463af1ebefbee83e84e29b51c0025f04091fab6
root@controlplane:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS   NAMES
628c38c58626   lerndevops/alpine:sleep  "/app/sleep"           34 seconds ago Up 33 seconds  cont4
ff4370d6cbb0   lerndevops/alpine:sleep  "/app/sleep"           43 seconds ago Up 42 seconds  cont3
d4cb2f98ff22   lerndevops/alpine:sleep  "/app/sleep"           33 minutes ago Up 33 minutes  cont2
alf8631c80ab   lerndevops/alpine:sleep  "/app/sleep"           33 minutes ago Up 33 minutes  cont1
```

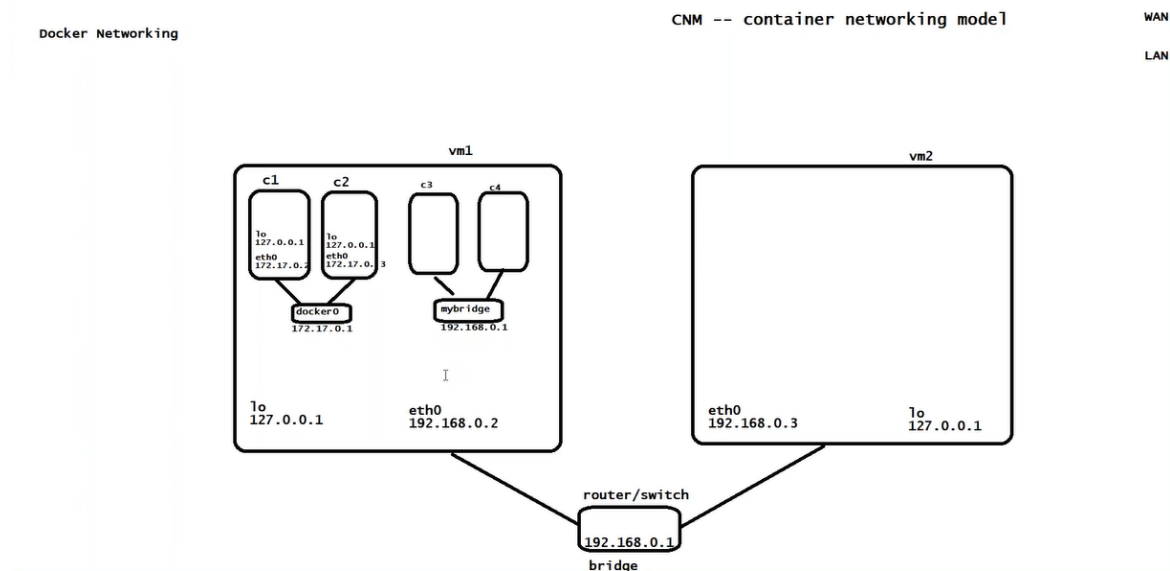
container in default network can communicate to each other but can't communicate to the containers in custom bridge network, because they are in different subnets.

```

root@controlplane:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
628c38c58626   lerndevops/alpine:sleep  "/app/sleep"           3 minutes ago Up 3 minutes   cont4
ff4370d6cbb0   lerndevops/alpine:sleep  "/app/sleep"           3 minutes ago Up 3 minutes   cont3
d4cb2f98ff22   lerndevops/alpine:sleep  "/app/sleep"           35 minutes ago Up 35 minutes  cont2
a1f9631c00ab   lerndevops/alpine:sleep  "/app/sleep"           35 minutes ago Up 35 minutes  cont1
root@controlplane:~# docker exec cont3 hostname -i
172.18.0.2
root@controlplane:~# docker exec cont4 hostname -i
172.18.0.3
root@controlplane:~# docker exec cont3 ping 172.18.0.3
PING 172.18.0.3 (172.18.0.3): 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=64 time=0.233 ms
64 bytes from 172.18.0.3: seq=1 ttl=64 time=0.225 ms
64 bytes from 172.18.0.3: seq=2 ttl=64 time=0.094 ms
^C
root@controlplane:~# docker exec cont3 ping 172.17.0.3
PING 172.17.0.3 (172.17.0.3): 56 data bytes

```

How containers in different networks communicate to each other?



UC: How can cont1 can communicate with cont4?

Solution: join the container in other network also, container can join different networks at same time.

```

root@node1:~# docker exec cont1 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1400  errors:0  dropped:0  overruns:0  frame:0
          TX packets:1385  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:0
          RX bytes:135108 (131.9 KiB)  TX bytes:133882 (130.7 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

docker network connect cont1 mybridge

Now cont1 have two IP addresses (eth0, eth1), because it's connected to two different networks.


```

root@node1:~# docker exec cont1 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1536  errors:0  dropped:0  overruns:0  frame:0
          TX packets:1521  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:0
          RX bytes:148212 (144.7 KiB)  TX bytes:146986 (143.5 KiB)

eth1      Link encap:Ethernet  HWaddr 02:42:C0:A8:00:04
          inet addr:192.168.0.4  Bcast:192.168.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:8  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:0
          RX bytes:656 (656.0 B)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

docker network disconnect cont1 mybridge

```

root@node1:~# docker network disconnect mybridge cont1
root@node1:~#
root@node1:~#
root@node1:~# docker exec cont1 ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:11:00:02
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1643  errors:0  dropped:0  overruns:0  frame:0
          TX packets:1628  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:0
          RX bytes:158586 (154.8 KiB)  TX bytes:157360 (153.6 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:0  errors:0  dropped:0  overruns:0  frame:0
          TX packets:0  errors:0  dropped:0  overruns:0  carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

```

How to create our own subnet ?

docker network create --help

docker network create --driver bridge mybridge1 --subnet=192.168.0.0/16

```

root@controlplane:~# docker network create --driver bridge mybridge1 --subnet=192.168.0.0/16
94042e9df9975a177de7328c993bd54c5d874ce66e64d2c0056688d3b4783007
root@controlplane:~# docker network ls
NETWORK ID        NAME          DRIVER  SCOPE
9d7d8349b98a     bridge       bridge  local
fde05e4476bc     host         host    local
24e23771db15     mybridge     bridge  local
64042e9df997     mybridge1    bridge  local
9c763a38595e     none        null    local
root@controlplane:~# docker inspect 94042e9df997
[
  {
    "Name": "mybridge1",
    "Id": "94042e9df9975a177de7328c993bd54c5d874ce66e64d2c0056688d3b4783007",
    "Created": "2021-01-26T16:24:44.739366473Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "192.168.0.0/16"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {},
    "Options": {},
    "Labels": {}
  }
]

```

create a container in mybridge1 network

docker run -d --name=cont5 --network mybridge1 lerndevops/alpine:sleep

docker run -d --name=cont6 --network mybridge1 lerndevops/alpine:sleep

```

root@controlplane:~# docker run -d --name=cont5 --network mybridge1 lerndevops/alpine:sleep
c286c3415715cba3fd553da19b14dfa623f18adf7825a30c79928f7954c411cc
root@controlplane:~# docker run -d --name=cont6 --network mybridge1 lerndevops/alpine:sleep
27761e35abf4033f2d654c53b6a87437c38f9827b583b5510b72e31bf070cbc2
root@controlplane:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
27761e35abf4   lerndevops/alpine:sleep   "/app/sleep"           7 seconds ago   Up 6 seconds   cont6
c286c3415715   lerndevops/alpine:sleep   "/app/sleep"           18 seconds ago   Up 18 seconds   cont5
628c38c58626   lerndevops/alpine:sleep   "/app/sleep"           25 minutes ago   Up 25 minutes   cont4
ff4370d6cbb0   lerndevops/alpine:sleep   "/app/sleep"           25 minutes ago   Up 25 minutes   cont3
d4cb2f98ff22   lerndevops/alpine:sleep   "/app/sleep"           57 minutes ago   Up 57 minutes   cont2
a1f8631c80ab   lerndevops/alpine:sleep   "/app/sleep"           57 minutes ago   Up 57 minutes   cont1
root@controlplane:~# docker exec cont5 hostname -i
192.168.0.2
root@controlplane:~# docker exec cont6 hostname -i
192.168.0.3
root@controlplane:~#

```

What is none network?

What happen if we switch off our wifi? None network

create a container in none network

docker run -d --name=cont7 --network none lerndevops/alpine:sleep

```

root@controlplane:~# docker network ls
NETWORK ID      NAME      DRIVER      SCOPE
9d7d8349b98a    bridge    bridge      local
rde05e4476bc    host      host        local
24e23771db15    mybridge  bridge      local
94042e9df987    mybridge1 bridge      local
9c763d38595e    none      null        local
root@controlplane:~# docker run -d --name=cont7 --network none lerndevops/alpine:sleep
d6dde186fa0dd5c87e3a6e6c2997314c8afecb34c296cec0a963a3ff2fc4ded8
root@controlplane:~# docker exec cont7 ifconfig
lo
  Link encap:Local Loopback
  inet addr:127.0.0.1  Mask:255.0.0.0
  UP LOOPBACK RUNNING  MTU:65536  Metric:1
  RX packets:0 errors:0 dropped:0 overruns:0 frame:0
  TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
  collisions:0 txqueuelen:1000
  RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
root@controlplane:~#

```

Cont7 is not a part of any network. No eth0 interface, not reachable to outside the world.

It's a independent process, apart from the default docker networking can use any network provider (CNI), not bounded with docker default (CNM).

What is the purpose of none network?

I am not joining any default docker networking. I can join any networking solution. Used by K8s to join CNI (weave).

This container is all alone container, none container can reach to this, within the local host can access, but from outside none container can access.

UC: Once pullout from local Wifi, can be connected to any network.

I don't want to use docker's default network driver, instead we want to install customized network drivers.

What is host network?

create a container in host network

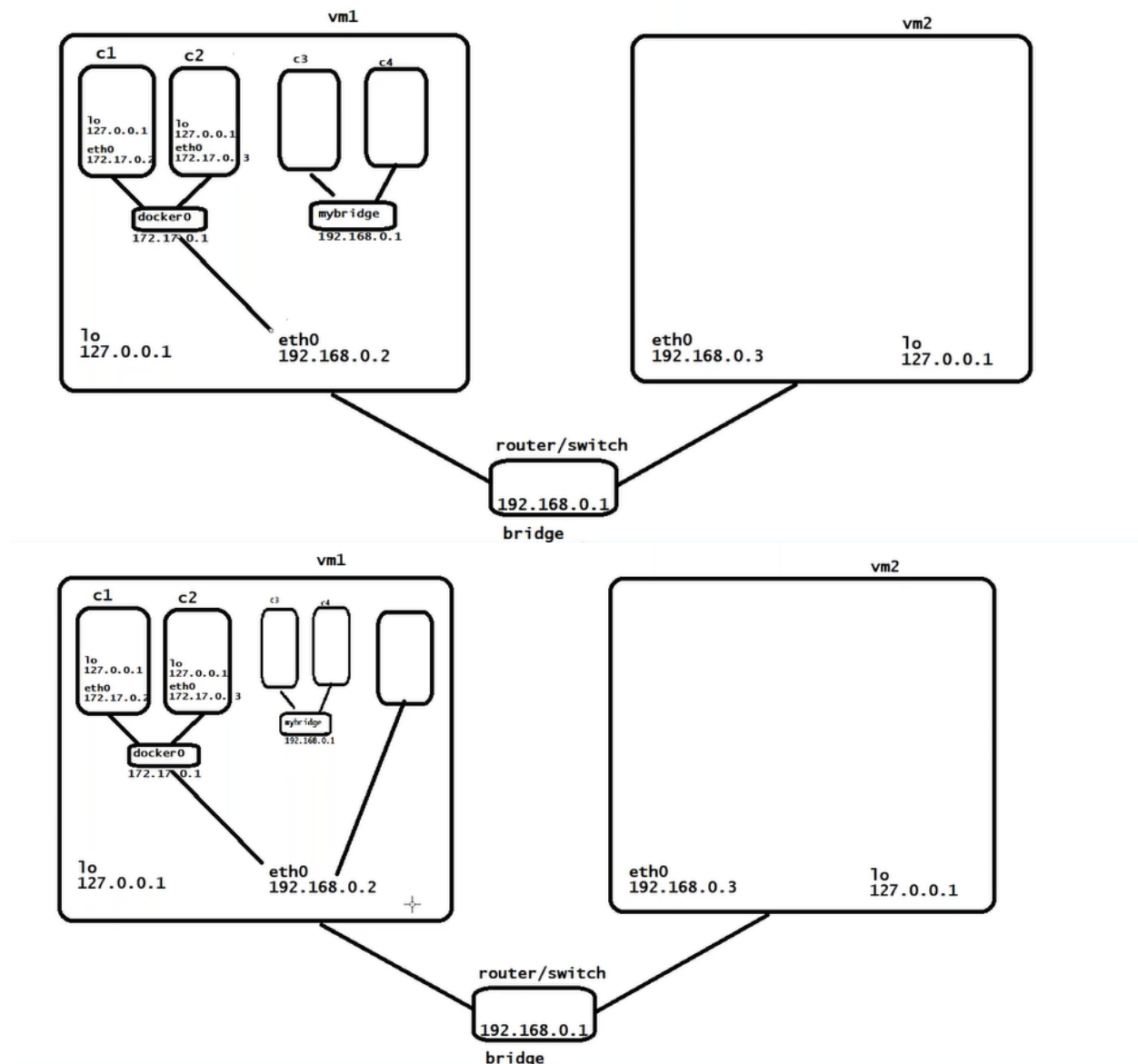
docker run -d --name=cont8 --network host lerndevops/alpine:sleep

```

root@controlplane:~# docker run -d --name=cont8 --network host lerndevops/alpine:sleep
3374588ca69bef80ceedb24888c3d44902ee7a9c6ea0f9b7570843a2947945b3
root@controlplane:~# hostname -i
10.150.0.6
root@controlplane:~# docker exec cont8 hostname -i
10.150.0.6
root@controlplane:~#

```

Hey! Container join the host network, there is no docker0 between container and host (port mapping not required).



This is not an isolated process now, it's a process of running on your server. Lose all the isolated feature of the container.
don't need to publish the port

```

root@node1:~# docker run -d --network host nginx
a94954561a2d3901c089243326da53b75089c3f343c258bfad6bafbc19f61683
root@node1:~#
root@node1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
a94954561a2d   nginx    "/docker-entrypoint..." 5 seconds ago  Up 5 seconds  -              magical_mayer
8215b8a8bde2   lerndevops/alpine:sleep  "/app/sleep"            About a minute ago  Up About a minute  -              cont6

```

If we try to access the same from browser, it will be accessible, if we try to run same second time?

```

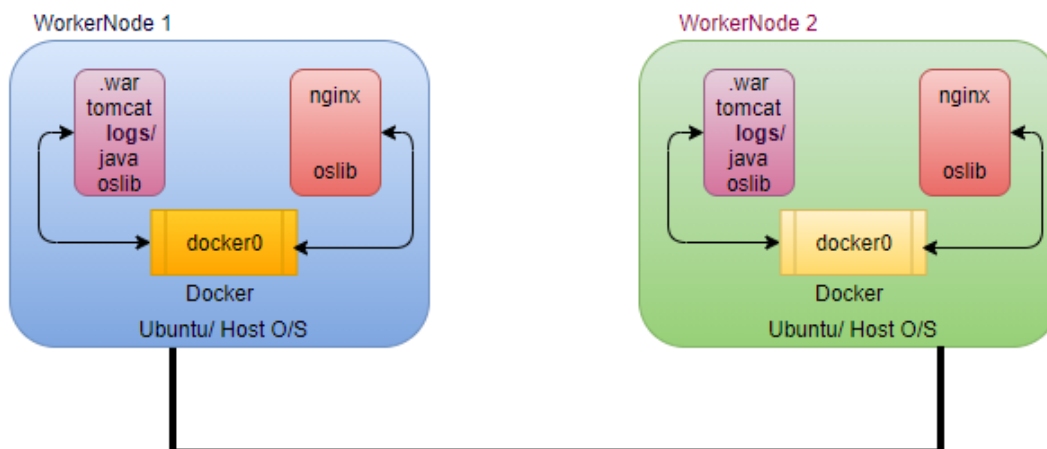
root@node1:~# docker run -d --network host nginx
a94954561a2d3901c089243326da53b75089c3f343c258bfad6bafbc19f61683
root@node1:~#
root@node1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
a94954561a2d   nginx    "/docker-entrypoint..." 5 seconds ago  Up 5 seconds  -              magical_mayer
8215b8a8bde2   lerndevops/alpine:sleep  "/app/sleep"            About a minute ago  Up About a minute  -              cont6
c8922e3ba8c0   lerndevops/alpine:sleep  "/app/sleep"            8 minutes ago    Up 8 minutes    -              cont5
88f6a9ab59e1   lerndevops/alpine:sleep  "/app/sleep"            17 minutes ago   Up 17 minutes   -              cont4
2b2458983fa1   lerndevops/alpine:sleep  "/app/sleep"            17 minutes ago   Up 17 minutes   -              cont3
2294d0fa3584   lerndevops/alpine:sleep  "/app/sleep"            26 minutes ago   Up 26 minutes   -              cont2
a30824ed75b1   lerndevops/alpine:sleep  "/app/sleep"            26 minutes ago   Up 26 minutes   -              cont1
root@node1:~#
root@node1:~# docker run -d --network host nginx
822c894809742241645d4f07ea0c89acee48e4c8b40900cc4b75bfeada888b1f
root@node1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
a94954561a2d   nginx    "/docker-entrypoint..." About a minute ago  Up About a minute  -              magical_mayer
8215b8a8bde2   lerndevops/alpine:sleep  "/app/sleep"            3 minutes ago     Up 3 minutes      -              cont6
c8922e3ba8c0   lerndevops/alpine:sleep  "/app/sleep"            10 minutes ago    Up 10 minutes     -              cont5
88f6a9ab59e1   lerndevops/alpine:sleep  "/app/sleep"            18 minutes ago    Up 18 minutes     -              cont4

```

port is already in used.

Limitation: As of now we have discussed the networking within the same host, which is not a realistic scenario.

How containers communicate across different node?



By default, nodes are able to communicate to each other through the underlay network (AWS, GCP)

Can container available in different nodes communicate to each other by default? No

Solution: We have to establish a cluster

In realistic environment we required cluster of at least 3 nodes.

To communicate across multiple docker hosts, we have to first initialize the cluster, go to any container orchestration solution e.g., docker swarm, k8s which manages the containers across the cluster.

Docker swarm:

- Container orchestration tool
- Container management tool

docker swarm comes with the docker installation, and by default it is in inactive state.

```
root@controlplane:~# docker info | grep -i swarm
Swarm: inactive
WARNING: No swap limit support
WARNING: No blkio weight support
WARNING: No blkio weight_device support
root@controlplane:~# docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
d122251982d0    bridge    bridge  local
2cea1736596d    host      host     local
109b25cc349d    none      null     local
root@controlplane:~#
```

How to activate the swarm?

docker swarm init

As we initialize a docker swarm, docker creates a new extra network which span across the cluster, when you join the multiple docker hosts on the single cluster, docker joins the multiple docker host in single virtual network again as an overlay network.

docker swarm init

In master creates Overlay network

```
root@controlplane:~# docker swarm init
Swarm initialized: current node (ueti2kf3w4f4343lavce64h6u) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-38pogv30lr3czfgcyn3pyhblpdgd4z3srliwtlvfthdwiwbjv0-1ltmlrlivp3nwg3f6utlphvz5g 10.128.0.4:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

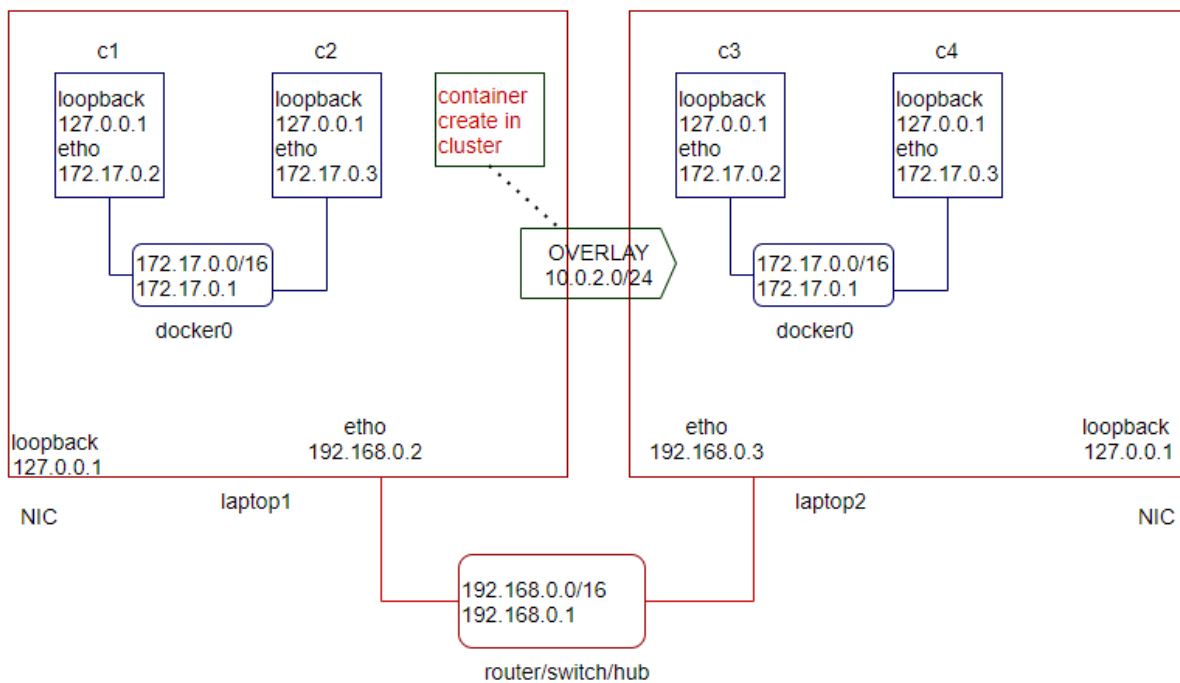
root@controlplane:~# docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
d122251982d0    bridge    bridge  local
d04b6fd37b43    docker_gwbridge  bridge  local
2cea1736596d    host      host     local
561sdfsgcgwx    ingress    overlay  swarm
109b25cc349d    none      null     local
root@controlplane:~#
```

docker inspect <networkId>

```

root@controlplane:~# docker inspect 56lsdfgsgwmx
[
  {
    "Name": "ingress",
    "Id": "56lsdfgsgwmxrn99a1317912a",
    "Created": "2021-01-29T13:48:15.578208494Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.0.0/24",
          "Gateway": "10.0.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": true,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "ingress-sbox": {
        "Name": "ingress-endpoint",
        "EndpointID": "73f18746e57857edd1599a05303dc46844202cc3b1dd8b34c60368c645bff23c",
        "MacAddress": "02:42:0a:00:00:02",
        "IPv4Address": "10.0.0.2/24",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.driver.overlay.vxlanid_list": "4096"
    },
    "Labels": {},
    "Peers": [
      {
        "Name": "16119258fe7f",
        "IP": "10.128.0.4"
      }
    ]
  }
]

```



In Worker Node01

```

root@node01:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d7fc0a0a1102        bridge              bridge              local
82549d5d2726        host                host                local
17bde456d8b5        none                null                local
root@node01:~# docker swarm join --token SWMTKN-1-38pogv30lr3czfgcyn3pyhblpdgd4z3srlwtlvfthdwiwbjv0-11tmrlivp3n
wg3f6utlphvz5g 10.128.0.4:2377
This node joined a swarm as a worker.
root@node01:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
d7fc0a0a1102        bridge              bridge              local
a89633fcd848        docker_gwbridge     bridge              local
82549d5d2726        host                host                local
561sdfsgcgwx        ingress              overlay             swarm
17bde456d8b5        none                null                local
root@node01:~#

```

In Worker Node02

```

root@node02:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
4fd0772cfc8a        bridge              bridge              local
4ab30da58a24        host                host                local
8ef648e8ed7a        none                null                local
root@node02:~# docker swarm join --token SWMTKN-1-38pogv30lr3czfgcyn3pyhblpdgd4z3srlwtlvfthdwiwbjv0-11tmrlivp3nwg3f6utlphvz5g
10.128.0.4:2377
This node joined a swarm as a worker.
root@node02:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
4fd0772cfc8a        bridge              bridge              local
9ec3892b5c20        docker_gwbridge     bridge              local
4ab30da58a24        host                host                local
561sdfsgcgwx        ingress              overlay             swarm
8ef648e8ed7a        none                null                local
root@node02:~#

```

In Master Node

```

root@controlplane:~# docker node ls
ID                                HOSTNAME                STATUS    AVAILABILITY    MANAGER STATUS    ENGINE VERSION
ueti2kf3w4f4343lavce64h6u *    controlplane            Ready     Active           Leader             20.10.2
scgrm38lqrrjzts1cbhrd1d0g        node01                   Ready     Active           -                  20.10.2
rxblo15tctsslgh5n4755klls        node02                   Ready     Active           -                  20.10.2
root@controlplane:~#

```

UC: how to regenerate the token?

docker swarm join-token worker

```

root@manager:~# docker swarm join-token worker
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-43mld103rwhll1a4f2fxa6dglulkjy8qvy75dllx4wjpg9kzak-dco1jfcjcdiacwwufjm5shu7zh 10.128.0.33:2377

root@manager:~#

```

single vm (container), cluster (service- group of containers)

docker service create --name myapp --replicas 7 -p 9000:3000

lerndevops/samplepyapp:v2 # group of containers called service

docker swarm service provides **horizontal scaling, self-healing and load balancing.**

How to create your custom overlay network?

docker network create myoverlay --driver overlay

```

root@controlplane:~# docker network create myoverlay --driver overlay
t452nijmbkxh2s4p2x0pwjwjg
root@controlplane:~# docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
d122251982d0    bridge    bridge   local
d04b6fd37b43    docker_gwbridge  bridge   local
2ceal736596d    host      host     local
56lsdfsgcgwx    ingress   overlay   swarm
t452nijmbkxh    myoverlay overlay   swarm
109b25cc349d    none      null      local
root@controlplane:~# █

```

How to create a service on custom overlay network?

`docker service create --name mysvc --network myoverlay --replicas=5`

`lerndevops/alpine:sleep`

```

root@controlplane:~# docker service create --name mysvc --network myoverlay --replicas=5 lerndevops/alpine:sleep
bd3glt0nsjhvj9e9k9h9w8rl9
overall progress: 5 out of 5 tasks
1/5: running [=====>]
2/5: running [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: running [=====>]
verify: Service converged
root@controlplane:~# █

```

`docker service ls`

`docker service ps <name of the svc>` # detailed info of service

In master node

```

root@controlplane:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS        NAMES
23fe358c1381   lerndevops/alpine:sleep  "/app/sleep"           4 minutes ago  Up 4 minutes                mysvc.3.j18iy1v8momx4n64qjojwmca3
root@controlplane:~# █

```

In worker node1

```

root@node01:~# docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS        NAMES
bb23c9645a3a   lerndevops/alpine:sleep  "/app/sleep"           5 minutes ago  Up 5 minutes                mysvc.1.p08px6np
irzdanph9eed4rydt
e840e669bbd3   lerndevops/alpine:sleep  "/app/sleep"           5 minutes ago  Up 5 minutes                mysvc.4.zm6p86yt
ujyp1fhpe5dpc1441
root@node01:~# docker network ls
NETWORK ID      NAME      DRIVER  SCOPE
d7fc0a0a1102    bridge    bridge   local
a89633fcd848    docker_gwbridge  bridge   local
82549d5d2726    host      host     local
56lsdfsgcgwx    ingress   overlay   swarm
t452nijmbkxh    myoverlay overlay   swarm
17bde456d8b5    none      null      local
root@node01:~# █

```

In worker node2


```

root@node02:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
87d3ad263281   lerndevops/alpine:sleep  "/app/sleep"           6 minutes ago Up 6 minutes   mysvc.5.jqf5ml7vau5eonx
zt49
199b01df4226   lerndevops/alpine:sleep  "/app/sleep"           6 minutes ago Up 6 minutes   mysvc.2.7f6wnlu696qzvq
oysl
root@node02:~# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
4fd0772cfc8a   bridge   bridge    local
9ec3892b5c20   docker_gwbridge   bridge    local
4ab30da58a24   host     host      local
56lsdfsgcgwx   ingress  overlay   swarm
t452nijmbkhh   myoverlay overlay   swarm
8ef648e8ed7a   none     null      local
root@node02:~#

```

UC: Container Communication across the cluster?

In Master

```

root@controlplane:~# docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
d122251982d0   bridge   bridge    local
d04b6fd37b43   docker_gwbridge   bridge    local
2ceaf736596d   host     host      local
56ladfsgcgwx   ingress  overlay   swarm
t452nijmbkhh   myoverlay overlay   swarm
109b25cc349d   none     null      local
root@controlplane:~# docker inspect t452nijmbkhh
[
  {
    "Name": "myoverlay",
    "Id": "t452nijmbkhh2s4p2x0pwjwjq",
    "Created": "2021-01-29T14:36:26.670788968Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.1.0/24",
          "Gateway": "10.0.1.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    }
  }
]

```

docker exec <container id> hostname -i

```

root@controlplane:~# docker service ls
ID            NAME    MODE     REPLICAS  IMAGE          PORTS
bd3glt0nsjvh mysvc   replicated 5/5        lerndevops/alpine:sleep
root@controlplane:~# docker service ps mysvc
ID            NAME    IMAGE          NODE    DESIRED STATE  CURRENT STATE    ERROR
PORTS
p08px6npirzd mysvc.1  lerndevops/alpine:sleep  node01  Running        Running 3 minutes ago
7f6wnlu696qz mysvc.2  lerndevops/alpine:sleep  node02  Running        Running 3 minutes ago
j18iylv8mcmx mysvc.3  lerndevops/alpine:sleep  controlplane  Running        Running 3 minutes ago
zm6p86ytujyp mysvc.4  lerndevops/alpine:sleep  node01  Running        Running 3 minutes ago
jqf5ml7vau5e mysvc.5  lerndevops/alpine:sleep  node02  Running        Running 3 minutes ago
root@controlplane:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS   NAMES
23fe358c1381   lerndevops/alpine:sleep  "/app/sleep"           4 minutes ago Up 4 minutes   mysvc.3.j18iylv8mcmx4n64qjojwmca3
root@controlplane:~# docker exec 23fe358c1381 hostname -i
10.0.1.5
root@controlplane:~#

```

In Node01

docker exec <container id> hostname -i

```

root@node01:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
bb23c9645a3a   lerndevops/alpine:sleep  "/app/sleep"           14 minutes ago Up 14 minutes        mysvc.1.p08px6
npirzdanph9eed4rydt
e840e669bbd3   lerndevops/alpine:sleep  "/app/sleep"           14 minutes ago Up 14 minutes        mysvc.4.zm6p86
ytujyp1fhpe5dpc1441
root@node01:~# docker exec bb23c9645a3a hostname -i
10.0.1.3
root@node01:~# docker exec e840e669bbd3 hostname -i
10.0.1.6
root@node01:~# █

```

In Node02

`docker exec <container id> hostname -i`

```

root@node02:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
87d3ad263281   lerndevops/alpine:sleep  "/app/sleep"           15 minutes ago Up 15 minutes        mysvc.5.jqf5ml
7vau5eonx5wfp9zt49
199b01df4226   lerndevops/alpine:sleep  "/app/sleep"           15 minutes ago Up 15 minutes        mysvc.2.7f6wnl
u696gzvqv18hnfwy1s
root@node02:~# docker exec 87d3ad263281 hostname -i
10.0.1.7
root@node02:~# docker exec 199b01df4226 hostname -i
10.0.1.4
root@node02:~# █

```

All containers are created under the same subnet, so they can ping to each other by default.

```

root@node01:~# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
bb23c9645a3a   lerndevops/alpine:sleep  "/app/sleep"           14 minutes ago Up 14 minutes        mysvc.1.p08px6
npirzdanph9eed4rydt
e840e669bbd3   lerndevops/alpine:sleep  "/app/sleep"           14 minutes ago Up 14 minutes        mysvc.4.zm6p86
ytujyp1fhpe5dpc1441
root@node01:~# docker exec bb23c9645a3a hostname -i
10.0.1.3
root@node01:~# docker exec e840e669bbd3 hostname -i
10.0.1.6
root@node01:~# docker exec e840e669bbd3 ping 10.0.1.5
PING 10.0.1.5 (10.0.1.5): 56 data bytes
64 bytes from 10.0.1.5: seq=0 ttl=64 time=1.115 ms
64 bytes from 10.0.1.5: seq=1 ttl=64 time=0.531 ms
64 bytes from 10.0.1.5: seq=2 ttl=64 time=0.711 ms
64 bytes from 10.0.1.5: seq=3 ttl=64 time=3.389 ms

```

UC: we have docker is installed on master as well as on the worker nodes, communication happens between docker engine and docker engine.

How to containers in different overlay network communicates to each-other?

`docker network create -help`

```
root@ip-172-31-21-252:~# docker network create --help
```

```
Usage: docker network create [OPTIONS] NETWORK
```

```
Create a network
```

```
Options:
```

--attachable	Enable manual container attachment
--aux-address map	Auxiliary IPv4 or IPv6 addresses used by Network driver (default map[])
--config-from string	The network from which copying the configuration
--config-only	Create a configuration only network
-d, --driver string	Driver to manage the Network (default "bridge")
--gateway strings	IPv4 or IPv6 Gateway for the master subnet
--ingress	Create swarm routing-mesh network
--internal	Restrict external access to the network
--ip-range strings	Allocate container ip from a sub-range
--ipam-driver string	IP Address Management Driver (default "default")
--ipam-opt map	Set IPAM driver specific options (default map[])
--ipv6	Enable IPv6 networking
--label list	Set metadata on a network
-o, --opt map	Set driver specific options (default map[])
--scope string	Control the network's scope
--subnet strings	Subnet in CIDR format that represents a network segment